AMRITA VISHWA VIDYAPEETHAM

DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF THE UGC ACT, 1956

TIFAC-CORE IN CYBER SECURITY

AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112

SOC Alert Prioritization Using Binary/multiclass Classification and Unsupervised Anomaly Detection.

24CY731

M.TECH CYBER SECURITY

SEMESTER: II

Mini Project Report Submitted by,

Reg. No	Name
CB.SC.P2CYS25024	Saurango Jyoti Khound
CB.SC.P2CYS25010	Kaleeswaran P

Table of Contents

Sl. No	Contents
1	Abstract
2	Introduction
3	Literature Survey
4	Problem Statement
5	Proposed Methodology
6	Performance Results
7	Conclusion & Future work
8	References

1. Abstract

Security Operations Centers (SOCs) generate a large volume of alerts from various monitoring tools, making it difficult for analysts to identify genuine threats. Most alerts are benign, leading to alert fatigue and delayed response to critical incidents. This project addresses the problem of efficient alert triage and prioritization using data mining and machine learning techniques.

A hybrid detection system is proposed that combines supervised and unsupervised learning approaches. A Random Forest classifier is used to perform binary classification of system events into benign and malicious categories, learning patterns from labeled data. In parallel, an Isolation Forest model is employed for unsupervised anomaly detection to identify unusual or previously unseen behaviors. The outputs of both models are integrated using a weighted risk scoring mechanism, where classification probability and anomaly score are combined to compute an overall risk score for each event. Based on this score, alerts are categorized into High, Medium, and Low priority levels.

The system is evaluated using the BETH dataset for binary classification and the UNSW-NB15 dataset for multiclass classification of attack types. The binary model achieves high performance with approximately 95% accuracy and strong recall for malicious events. The multiclass model provides detailed attack categorization with moderate performance, achieving around 73% accuracy.

The proposed approach demonstrates that combining classification and anomaly detection with risk-based prioritization can significantly improve alert management and help analysts focus on the most critical threats.

2. Introduction

In today's digital era, organizations rely heavily on interconnected systems, cloud infrastructures, and networked applications, making cybersecurity a critical aspect of modern computing. With the rapid growth of data and increased connectivity, cyber threats have become more frequent, sophisticated, and difficult to detect. Attackers continuously evolve their techniques, making traditional security mechanisms insufficient to handle modern threats effectively. As a result, ensuring timely detection and response to malicious activities has become a major challenge for security teams.

Security Operations Centres (SOCs) play a crucial role in monitoring, detecting, and responding to security incidents. However, these systems generate an overwhelming number of alerts from various tools such as intrusion detection systems, endpoint monitoring solutions, and log analysis platforms. A significant portion of these alerts are benign or false positives, which leads to inefficiencies in threat detection. This problem necessitates intelligent solutions that can automatically analyse, classify, and prioritize alerts to assist analysts in focusing on the most critical threats.

2.1 Overview of Cyber Security Problem

One of the primary challenges in cybersecurity is the **sheer volume and complexity of security data** generated in real-time environments. Every system activity, such as file access, process execution, network communication, and system calls, produces logs that need to be analysed. SOC analysts are required to monitor thousands of such events daily, making manual analysis impractical.

A major issue arising from this is **alert fatigue**, where analysts are overwhelmed by the number of alerts and may overlook or delay the investigation of critical threats. Additionally, many traditional detection systems rely on signature-based or rule-based approaches, which are effective only for known attack patterns. These systems fail to detect **zero-day attacks** or novel malicious behaviours that do not match predefined rules.

Another key problem is the lack of effective **alert prioritization mechanisms**. In many systems, all alerts are treated with equal importance, forcing analysts to spend time on low-risk events while potentially missing high-risk threats. Therefore, there is a strong need for automated solutions that can not only detect malicious activities but also rank them based on their severity.

2.2 Role of AI in Cyber Security

Artificial Intelligence (AI) and Machine Learning (ML) have significantly transformed the field of cybersecurity by enabling automated and intelligent threat detection. Unlike

traditional rule-based systems, ML models can learn patterns from historical data and generalize to unseen scenarios.

Supervised learning techniques, such as classification algorithms, are widely used to identify known attack patterns by learning from labelled datasets. These models can accurately distinguish between benign and malicious activities based on features extracted from system logs or network traffic. However, they are limited to detecting attacks that are similar to those seen during training.

To overcome this limitation, unsupervised learning techniques are employed for **anomaly detection**. These methods learn the normal behaviour of a system and identify deviations from this behaviour as potential threats. This is particularly useful for detecting unknown or zero-day attacks.

By combining supervised and unsupervised approaches, AI-based systems can provide a more robust and comprehensive solution for cybersecurity. Additionally, AI enables automation, scalability, and faster response times, making it an essential component of modern SOC operations.

2.3 Motivation for the Project

The motivation for this project stems from the practical challenges faced by SOC analysts in handling large volumes of alerts. The traditional approach of manually analyzing alerts is time-consuming, inefficient, and prone to human error. Analysts often spend significant time investigating low-risk or false-positive alerts, which reduces their ability to focus on genuine threats.

Furthermore, existing systems often lack an integrated mechanism to combine multiple sources of information, such as classification results and anomaly detection outputs, into a unified decision-making framework. This creates a gap between detection and actionable intelligence.

To address these challenges, there is a need for a system that can:

- Automatically detect malicious activities,
- Identify unusual behavior patterns,
- Assign a meaningful risk score to each event, and
- Prioritize alerts based on their severity.

This project aims to bridge this gap by developing a hybrid detection and alert prioritization system that improves the efficiency and effectiveness of security monitoring.

2.4 Objectives

The primary objectives of this project are:

- To develop a system that classifies system events as **benign or malicious using supervised learning techniques**
- To extend the analysis to **multiclass classification**, enabling identification of specific attack types
- To implement **unsupervised anomaly detection** for identifying unusual or previously unseen behaviors
- To design a **risk scoring mechanism** that combines classification probability and anomaly score into a unified metric
- To categorize alerts into **High, Medium, and Low priority levels** based on the computed risk score
- To reduce analyst workload by providing **automated alert prioritization and ranking**
- To evaluate the system using real-world datasets such as **BETH (binary classification)** and **UNSW-NB15 (multiclass classification)**

3. Literature Survey

Ref.	Author(s) & Year	Technique / Model	Key Contributions	Advantages	Limitations
[1]	Gelman *et al.* (2023)	Supervised ML (TEQ)	Predicts alert actionability and filters low-priority alerts	Reduces response time (~22.9%), suppresses false positives (54%)	Requires labeled data; possible overfitting
[2]	Ghadermazi *et al.* (2024)	ML + Optimization	Clustering alerts and assigning analysts based on expertise	Reduces alert backlog by 60.16%	Depends on accurate expertise modeling; risk of misclassification
[3]	O'Brien *et al.* (2020)	Decision Tree, Random Forest	Alert prioritization using traffic data	High accuracy (<2% error); simple implementation	Cannot handle zero-day attacks; relies on labeled data
[4]	Jalalvand *et al.* (2024)	Survey (HAT Model)	Categorizes alert prioritization methods; Human-AI teaming	Highlights strengths of AI scalability and human insight	No new model proposed
[5]	Jalalvand *et al.* (2025)	Deep RL (L2DHF)	Learns when to defer alerts using human feedback	Improves accuracy (13–67%); reduces misprioritization	High complexity; depends on feedback quality
[6]	Chowdhury & Tanvir (2026)	Trust Calibration Framework	Aligns model confidence with cost-sensitive decisions	Reduces cost-weighted loss; model-agnostic	Requires proper calibration and cost modeling
[7]	Wang *et al.* (2024)	Reinforcement Learning (AlertPro)	Uses contextual and historical features for multi-step attacks	Improves detection (~30%); real-time ranking	Requires tuning; partial feedback limitation
[8]	Singh (2025)	Hybrid LLM + ML	Combines LLM with RF and threat intelligence	Adds contextual understanding to alerts	No experimental validation yet
[9]	Baha & Saci (2024)	Review (AI-based methods)	Rule-based, ML, DL approaches	Highlights accuracy vs interpretability trade-off	No implementation
[10]	Wiik Eckhoff *et al.* (2025)	Graph-based Model	Uses graph matching for alert contextualization	Detects multi-stage attack patterns effectively	Computational complexity

4. Problem Statement

4.1 Problem Definition:

The problem addressed in this work is the **accurate and efficient detection of anomalies (or malicious activities) in complex systems such as networks, databases, or user behavior logs using machine learning techniques**. Traditional rule-based or signature-based systems fail to detect unknown or evolving threats, making it necessary to design intelligent systems that can automatically identify abnormal patterns from large-scale data.

4.2 Why It Is Important

- **Increasing Cyber Threats:** With the rapid growth of digital systems, cyberattacks such as intrusions, fraud, and data breaches are increasing in both frequency and sophistication.
- **Limitations of Traditional Systems:** Signature-based systems can only detect known attacks and are ineffective against zero-day threats.
- **Data Explosion:** Modern systems generate massive volumes of data, making manual monitoring impractical.
- **Real-time Detection Requirement:** Delayed detection can lead to severe financial and reputational damage.

Hence, there is a critical need for **automated, scalable, and intelligent anomaly detection systems** that can identify both known and unknown threats in real time.

4.3 Current Challenges

- **High False Positives:** Many machine learning models incorrectly classify normal behavior as anomalous, leading to alert fatigue.
- **Imbalanced Data:** Anomalies are rare compared to normal data, making model training difficult.
- **Dynamic Environments:** Patterns of normal and abnormal behavior change over time, requiring adaptive models.
- **Interpretability Issues:** Many advanced models (e.g., deep learning) act as black boxes, making it hard to explain decisions.
- **Scalability:** Processing large-scale, high-dimensional data efficiently remains a challenge.
- **Feature Engineering Complexity:** Extracting meaningful features from raw data is non-trivial and impacts performance significantly.

5. Proposed Methodology

The methodology of this project is divided into clear and logical steps, starting from data collection and ending with alert prioritization and evaluation.

1. **Problem Formulation**
2. **Data Loading and Exploration**
3. **Data Cleaning**
4. **Feature Engineering and Preprocessing**
5. **Train–Test Split**
6. **Handling Class Imbalance**
7. **Binary Classification Model**
8. **Unsupervised Anomaly Detection**
9. **Risk (Alert) Score Calculation**
10. **Alert Prioritization**
11. **Error Analysis and Discussion**

Assumption: The methodology assumes a fixed weighting scheme of 70:30 in the risk score calculation, reflecting the relative importance assigned to labelled (supervised) data and unlabelled (anomaly-based) data.

Step 1: Problem Formulation

The objective of this work is to address alert overload in Security Operations Centers (SOCs) by prioritizing security alerts based on their risk level. Since most alerts are benign, analysts face difficulty identifying critical threats.

This problem is formulated as a hybrid learning task:

- A binary classification task to determine whether an event is malicious or benign.
- An anomaly detection task to identify unusual or rare behaviour.

The final goal is to compute a risk score for each event and rank alerts, enabling efficient investigation of high-risk events.

Step 2: Data Loading and Initial Exploration

The system uses the BETH dataset, consisting of separate labeled training and testing files:

- labelled_training_data.csv
- labelled_testing_data.csv

The datasets are loaded using Pandas, and their structure is examined by:

- Checking dataset dimensions (shape)
- Inspecting available features

- Understanding label-related columns (sus, evil)

This step ensures familiarity with the dataset before preprocessing.

Step 3: Data Cleaning and Label Construction

Instead of general cleaning techniques, the following specific operations (as per code) are performed:

- Removal of irrelevant columns:
    - timestamp, stackAddresses, args
- Construction of a binary label:
    - An event is labeled as malicious (1) if either sus = 1 or evil = 1
    - Otherwise, it is labeled as benign (0)`
- Dropping original label columns (sus, evil) after merging

This step simplifies the dataset and prepares it for binary classification.

Step 4: Feature Engineering and Preprocessing

Feature engineering is performed to capture meaningful behavioral patterns.

a) Composite Feature Creation

- A new feature user_process is created by combining:
    - userId and processName
- This helps capture user-specific process behavior.

b) Frequency Encoding

- Applied to:
    - processId, eventName, processName
- Each category is replaced with its normalized frequency in the training data
- Test data values not present in training are assigned 0

c) Preprocessing Pipeline

A ColumnTransformer-based pipeline is used:

- Numerical features → Standardized using StandardScaler
- Categorical features → Encoded using OneHotEncoder
    - Handles rare categories using min_frequency=5
    - Handles unseen categories using "infrequent_if_exist"

This ensures consistent and automated preprocessing.

Step 5: Train–Test Separation

The dataset is already provided as separate training and testing sets:

- X_train, y_train
- X_test, y_test

This eliminates the need for manual splitting and ensures unbiased evaluation.

Step 6: Handling Class Imbalance

Instead of explicit resampling techniques, class imbalance is handled using:

- Class weighting in Random Forest:
    - class_weight = "balanced_subsample"

This adjusts the importance of minority (malicious) samples during training.

Step 7: Binary Classification Model

A Random Forest Classifier is used as the supervised model.

Model Characteristics

- 250 decision trees (n_estimators=250)
- Maximum depth of 25
- Balanced sampling for handling class imbalance
- Parallel processing enabled

Output

The model outputs a probability score: $P(\text{malicious})$

A custom threshold of 0.25 is applied (instead of default 0.5) to increase sensitivity toward detecting malicious events.

Step 8: Unsupervised Anomaly Detection

An Isolation Forest model is used for anomaly detection.

Key Characteristics

- Trained on preprocessed training data
- Contamination level set to 3%
- Detects anomalies based on data isolation

Output

- Produces anomaly scores using decision_function
- Scores are:
    - Inverted
    - Normalized to range [0, 1]

Higher values indicate more anomalous behavior.

Step 9: Risk (Alert) Score Calculation

The outputs from both models are combined into a unified risk score:

$$\boldsymbol{Risk\ Score = \alpha \times P(malicious) + \beta \times Anomaly\ Score}$$

Where:

- $P(\text{malicious})$: Output from the Random Forest model
- Anomaly Score: Output from Isolation Forest

The supervised model is given higher weight (0.7) as it captures known attack patterns, while anomaly detection (0.3) helps identify unusual behavior.

Step 10: Alert Prioritization

Alerts are prioritized based on the computed risk score:

- Events are sorted in descending order of risk score
- Priority levels are assigned:
    - High: ≥ 0.8
    - Medium: 0.5 – 0.8
    - Low: < 0.5

Additionally:

- Duplicate alert patterns are removed based on:
    - processName, eventName

This produces a ranked list of alerts similar to real SOC workflows.

Step 11: Evaluation and Error Analysis

Model performance is evaluated using:

- Classification Report:
    - Precision, Recall, F1-score
- Confusion Matrix:
    - True positives, false positives, etc.

Error analysis focuses on:

- Misclassified events (false positives and false negatives)
- Understanding model weaknesses
- Improving threshold selection and feature design

6. Performance Results

6.1 Test Bed Configuration

The experiments were conducted in a standard computing environment with the following setup:

- **Platform:** Python (Jupyter Notebook)

- **Libraries Used:**
  - Scikit-learn
  - XGBoost
  - Pandas
  - NumPy
- **Hardware:** Standard CPU-based system (no GPU required)
- **Operating Environment:** Desktop-based execution

6.2 Dataset Details

1. BETH Dataset (Binary Classification)

- Contains system-level logs such as:
  - Process activity
  - System calls (eventName)
- Used for:
  - Binary classification (Benign vs Malicious)
  - Anomaly detection

2. UNSW-NB15 Dataset (Multiclass Classification)

- Contains network traffic data with multiple attack categories:
  - Normal, Generic, Exploits, Reconnaissance, Backdoor, Critical
- Used for:
  - Multiclass classification of attack types

6.3 Evaluation Metrics

The system performance is evaluated using standard classification metrics:

- **Accuracy:** Overall correctness of predictions
- **Precision:** Correctness of positive predictions
- **Recall:** Ability to detect actual positive cases

- **F1-score:** Balance between precision and recall

Metric	Value
Accuracy	95%
Precision (Malicious)	0.97
Recall (Malicious)	0.98
F1-score (Malicious)	0.97

Fig 1: Performance of Binary Classification

- The model achieves **high accuracy (95%)**, indicating strong overall performance.
- **Recall of 0.98** for the malicious class shows that the system is highly effective in detecting actual attacks, minimizing false negatives.
- **Precision of 0.97** indicates that most predicted malicious events are correct, reducing false positives.
- The high F1-score confirms a **balanced and reliable model**.

	processId	processName	eventName	Risk_Score	Priority
1652	879	systemd	getdents64	0.985999	High
1700	879	systemd	sched_process_exit	0.951403	High
1484	879	systemd	lstat	0.949913	High
1379	879	systemd	stat	0.948832	High
1486	879	systemd	security_inode_unlink	0.935958	High
1475	879	systemd	security_file_open	0.932669	High
1441	879	systemd	openat	0.927742	High
1487	879	systemd	unlink	0.920003	High
1460	879	systemd	fstat	0.918562	High
1791	885	(sd-pam)	sched_process_exit	0.889323	High
1683	879	systemd	close	0.887436	High
17596	7329	run-parts	getdents64	0.779283	Medium
17580	7329	run-parts	access	0.749090	Medium
17460	7328	sh	clone	0.741429	Medium
24161	7351	run-parts	security_bprm_check	0.731568	Medium
17585	7329	run-parts	security_file_open	0.719759	Medium
17446	7328	sh	access	0.717884	Medium
29772	7328	sh	sched_process_exit	0.709097	Medium
17601	7329	run-parts	clone	0.708696	Medium
29769	7329	run-parts	sched_process_exit	0.706660	Medium
29731	7329	run-parts	stat	0.705584	Medium
17583	7329	run-parts	fstat	0.702695	Medium
17447	7328	sh	security_file_open	0.701643	Medium
24159	7351	run-parts	execve	0.701407	Medium
17466	7329	sh	security_bprm_check	0.695119	Medium
17598	7329	run-parts	close	0.679597	Medium
17590	7329	run-parts	openat	0.677771	Medium
17449	7328	sh	fstat	0.674521	Medium
17448	7328	sh	openat	0.671223	Medium
17461	7329	sh	execve	0.658796	Medium
29771	7328	sh	close	0.654396	Medium
32523	7425	sh	stat	0.650101	Medium
29770	7328	sh	dup2	0.637729	Medium

Fig 2: Output showing the system successfully ranks alerts based on risk score, ensuring that high-risk events are prioritized.

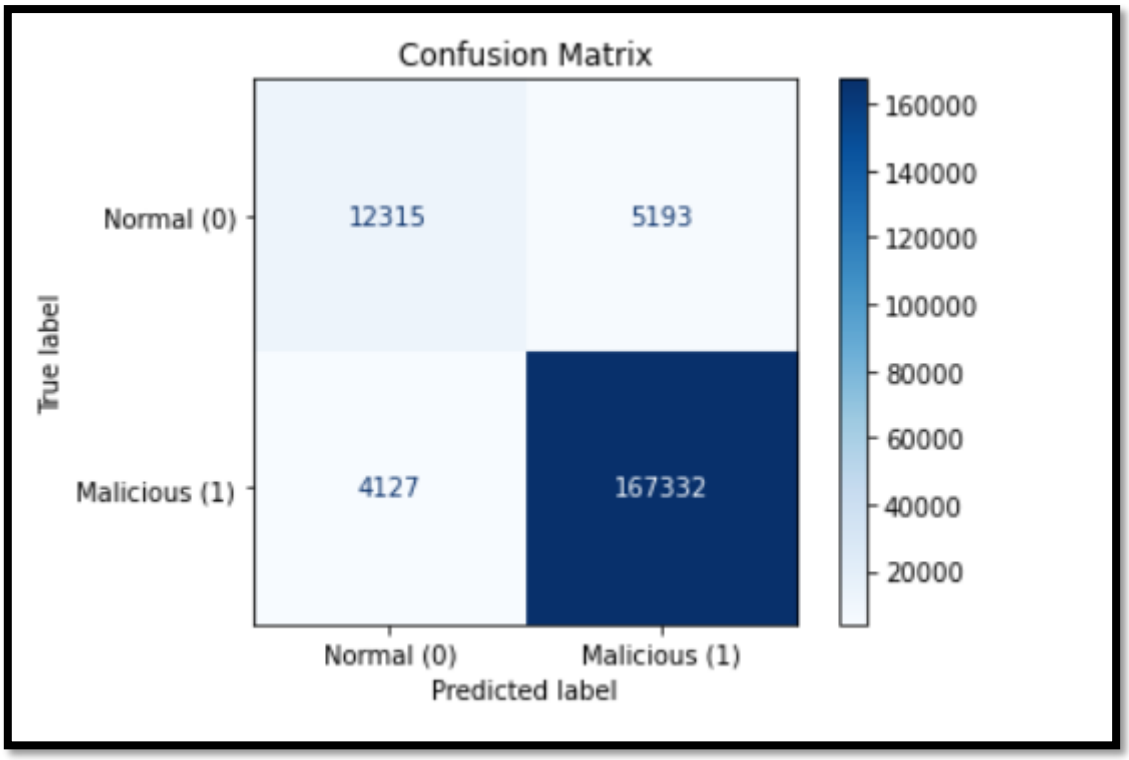

Fig 3: Confusion Matrix of Binary Classification

```
=== Classification Report ===
              precision    recall  f1-score   support

           0       0.75      0.70      0.73     17508
           1       0.97      0.98      0.97    171459

    accuracy                           0.95    188967
   macro avg       0.86      0.84      0.85    188967
weighted avg       0.95      0.95      0.95    188967
```

Fig 4: Classification report of binary classification.

Metric	Value
Accuracy	73%
Macro Recall	0.62
Weighted F1-score	0.77

Fig 5: Multiclass classification results

Class	Precision	Recall	F1-score
Normal	0.97	0.71	0.82
Generic	1.00	0.97	0.98
Exploits	0.76	0.64	0.70
Reconnaissance	0.89	0.83	0.86
Backdoor	0.07	0.44	Low
Critical	0.13	0.36	Low

Fig 6: Classwise performance of multiclass classification.

- The model achieves **moderate accuracy (73%)**, which is expected for a multiclass problem.
- High performance is observed in **Normal and Generic classes**, indicating the model can effectively identify common patterns.
- Moderate performance in **Exploits and Reconnaissance** suggests partial overlap between attack behaviors.

Multiclass classification is inherently **more complex than binary classification** due to: Increased number of classes and Overlapping feature patterns. The model provides **useful attack categorization**, even though performance is lower than binary detection.

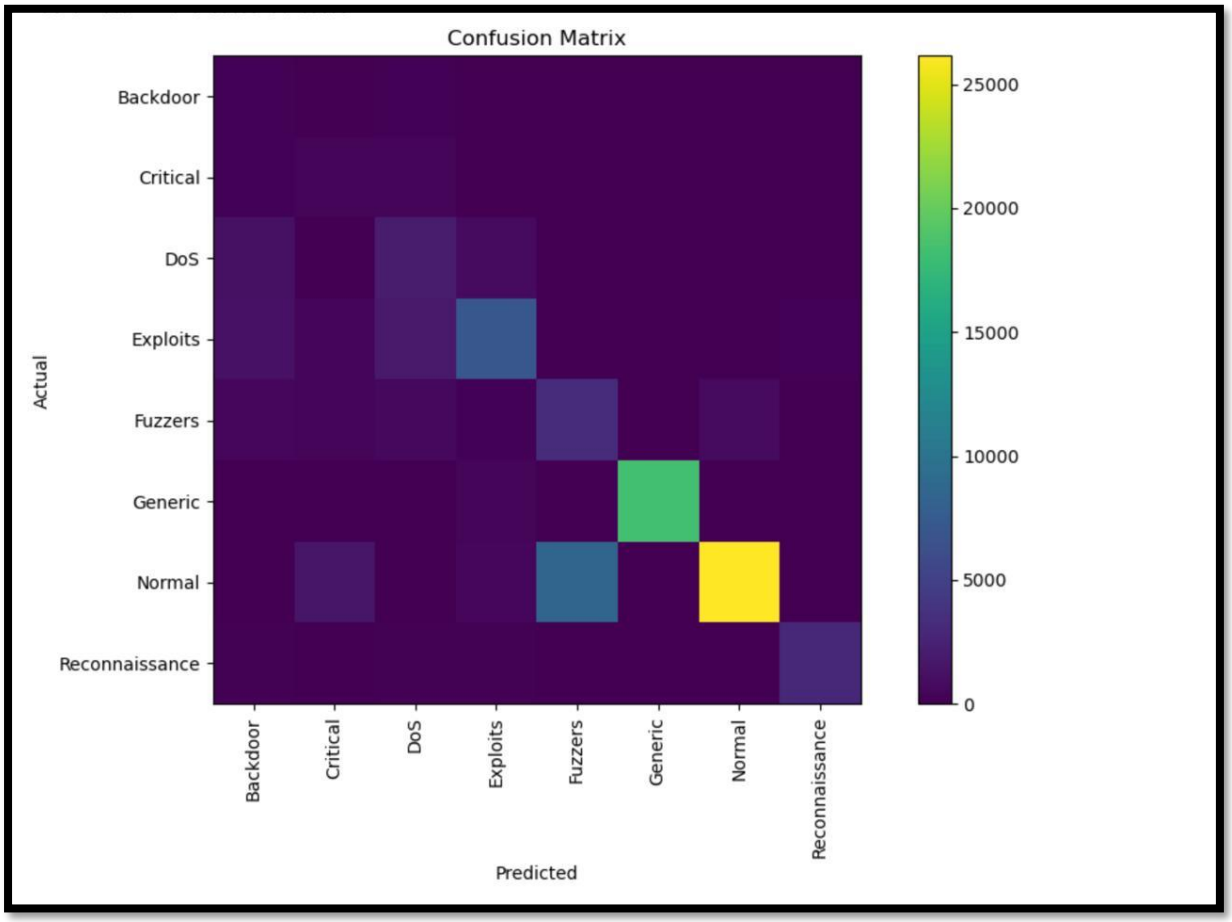

Fig 7: Confusion Matrix of Multiclass classification.

Aspect	**Binary (BETH)**	**Multiclass (UNSW-NB15)**
Task	Detection	Attack classification
Accuracy	95%	73%
Complexity	Low	High
Performance	Strong	Moderate
Use Case	Threat detection	Attack analysis

Fig 8: Comparative Analysis

- Binary classification is **more reliable for detection**, while multiclass classification provides **deeper insights into attack types**.
- The combination of both approaches enhances the **overall effectiveness of the system**.

7. Conclusion

This project addressed the challenge of **efficient threat detection and alert prioritization** in cybersecurity environments where large volumes of system logs generate numerous alerts. A hybrid machine learning framework was developed by integrating **supervised classification** and **unsupervised anomaly detection** to improve both detection accuracy and decision-making.

The proposed system utilized a **Random Forest classifier** to identify known malicious patterns and an **Isolation Forest model** to detect anomalous or previously unseen behaviors. The outputs of these models were combined using a **risk scoring mechanism**, which assigns a quantitative measure of threat severity to each event. Based on this score, alerts were categorized into **High, Medium, and Low priority levels**, enabling efficient alert triage.

The system was evaluated using the **BETH dataset** for binary classification and the **UNSW-NB15 dataset** for multiclass classification. The binary model achieved strong performance with approximately **95% accuracy**, high precision, and high recall for malicious events, demonstrating its effectiveness in detecting threats. The multiclass model achieved moderate performance (around **73% accuracy**), which is expected due to increased complexity and class imbalance. Despite this, it provided valuable insights into specific attack categories.

Overall, the project demonstrates that combining classification and anomaly detection with a risk-based prioritization framework can significantly enhance the efficiency of Security Operations Centers (SOCs) by reducing alert fatigue and enabling analysts to focus on high-risk threats.

7.1 Future Work

The proposed system provides an effective framework for alert detection and prioritization; however, several enhancements can further improve its performance and practical applicability.

Improvements in model design can make the system more adaptive and robust. Instead of using fixed weights for risk scoring, dynamic weighting based on model confidence or contextual factors can provide more accurate prioritization. Additionally, advanced techniques for handling class imbalance, such as cost-sensitive learning or synthetic data generation, can improve detection of rare attack instances. Incorporating richer, domain-specific features like temporal behavior, sequences of system calls, and user activity patterns can further enhance model performance.

The use of advanced AI models can significantly improve the system's capability to capture complex patterns. Deep learning approaches such as LSTM or Transformer-based models can better model sequential and temporal dependencies in security data. Graph-based learning techniques can represent relationships between users, processes, and events, enabling detection of multi-stage attacks. Furthermore, ensemble methods that combine multiple models can improve overall robustness and accuracy.

Real-time deployment is an important extension for practical SOC environments. The system can be adapted to process streaming data, enabling continuous monitoring and immediate alert generation. Integration with existing SOC tools such as SIEM platforms, along with interactive dashboards, can facilitate real-time visualization and decision-making for analysts.

Explainability and interpretability are critical for operational adoption. Incorporating explainable AI techniques can provide clear reasoning behind model predictions, helping analysts understand why an alert was flagged. This improves trust in the system and supports more informed decision-making.

Feedback-based learning can make the system adaptive over time. By incorporating analyst feedback on alerts (e.g., true positives and false positives), the system can continuously update and refine its models. This creates a human-in-the-loop framework that improves performance in dynamic environments.

Finally, the proposed framework can be extended to other domains beyond cybersecurity. The same approach can be applied to areas such as fraud detection, insider threat detection, and network intrusion detection, where prioritization of high-risk events is equally important.

8. References

[1] A. Gelman *et al.*, "That Escalated Quickly (TEQ): A Supervised ML Framework for Predicting Alert and Incident-Level Actionability," 2023. TEQ scores alerts using learned classifiers to filter low-actionability alerts. It reduces response time (~22.9%), suppresses 54% false positives (95.1% detection), and cuts alerts per incident by 14%. Limitations include reliance on labeled data and potential overfitting.

[2] M. Ghadermazi *et al.*, "A Machine Learning and Optimization Framework for CSOC Alert Management," 2024.This framework automates benign alert removal, clusters alerts, and assigns analysts. It reduces alert backlog by 60.16% in 8 hours. Limitations include dependency on accurate expertise modeling and risk of misclassification.

[3] K. O'Brien *et al.*, "Alert Prioritization and Strengthening Using Machine Learning," 2020.Uses decision trees and random forests for IDS alert prioritization with <2% error on CSE-CIC-IDS2018 dataset. Limitations include reliance on labeled data and lack of zero-day adaptability.

[4] A. Jalalvand *et al.*, "A Survey of SOC Alert Prioritization Techniques," 2024. Highlights Human-AI Teaming (HAT) and categorizes methods based on automation levels. Identifies gaps in adaptive and collaborative systems.

[5] A. Jalalvand *et al.*, “Learning to Defer with Human Feedback (L2DHF) for Alert Prioritization,” 2025. A DRLHF-based framework that improves prioritization accuracy (13–67%) and reduces misprioritization errors. Limitations include RL complexity and dependency on feedback quality.

[6] M. Chowdhury and S. Tanvir, “Decision-Aware Trust Signal Alignment for Alert Triage,” 2026. Introduces calibration of model confidence with cost-sensitive thresholds. Reduces cost-weighted loss but depends on accurate calibration and cost specification.

[7] H. Wang *et al.*, “AlertPro: Reinforcement Learning-Based Alert Prioritization for Multi-Step Attacks,” 2024. Uses contextual and historical features for prioritization. Improves attack discovery by ~30% with real-time re-ranking (<0.5s). Limitations include partial feedback usage and RL complexity.

[8] R. Singh, “Foundation-Sec-8B: A Hybrid LLM and ML Framework for Alert Prioritization,” 2025. Combines LLM with Random Forest and contextual threat intelligence. Currently conceptual with no empirical results.

[9] M. Baha and F. Saci, “AI-Based Correlation and Prioritization: A Review,” 2024. Compares rule-based, ML, and DL approaches. Highlights trade-off between accuracy and explainability and emphasizes need for interpretable systems.

[10] T. Wiik Eckhoff *et al.*, “Graph-Based Alert Contextualisation for Multi-Stage Attack Detection,” 2025. Uses graph matching networks to correlate alerts with historical attack patterns. Effective for contextualization but computationally complex.